

Recitation 2: Basics of Linear Algebra/Matrix Calculus

Presented by Amanda Wang

New slides, check for mistakes?

Caltech CS/CNS/EE 155

Winter 2026

Content Overview

Part I: Linear Algebra

- ▶ Matrices/vectors; multiplication; transpose/trace/inverse/det
- ▶ **Linear systems:** invertibility equivalences; rank tests; solvability/uniqueness
- ▶ **Subspaces:** $\text{Col}(A)$, $\mathcal{N}(A)$, $\text{Row}(A)$; rank-nullity; orthogonality
- ▶ Special matrices: symmetric/orthogonal; **PSD/PD** and spectral theorem
- ▶ Projections and least squares; norms, matrix norms, conditioning
- ▶ Eigenvalues/eigenvectors; **SVD** and pseudoinverse

Part II: Matrix Calculus

- ▶ Gradient/Hessian/Jacobian; layout conventions
- ▶ Differentials + trace trick; chain rule; structured-matrix caution
- ▶ Inverse/log-det differentials; least-squares gradient derivations

Matrices and Vectors

- ▶ A matrix $A \in \mathbb{R}^{m \times n}$ is a m by n rectangular array of numbers.
- ▶ A column vector $\mathbf{x}_C \in \mathbb{R}^m$.
- ▶ A row vector $\mathbf{x}_R \in \mathbb{R}^n$.

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad \mathbf{x}_C = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}, \quad \mathbf{x}_R = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}.$$

Matrices and Vectors (code)

In Numpy:

► Matrix:

```
A = np.array([[1, 2], [3, 4]])
```

► Vector:

```
np.array([1, 2, 3]).reshape((3,1)) # Column Vector  
# array([[1],  
#        [2],  
#        [3]])  
  
np.array([1, 2, 3]).reshape((1,3)) # Row Vector  
# array([[1, 2, 3]])
```

Matrix Multiplication

- If $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, then $C \in \mathbb{R}^{m \times p}$ is $C = AB$ with

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

Properties:

- Associativity: $(AB)C = A(BC)$
- Distributivity: $A(B + C) = AB + AC$
- No commutativity (in general): $AB \neq BA$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \\ b_{41} & b_{42} & b_{43} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \end{bmatrix}$$

Matrix Multiplication (Code)

For matrix multiplication, `np.dot` is the best option. Note that `np.multiply` is not matrix multiplication, but rather element-wise multiplication. Also, `np.matrix` is unwieldy for > 2 dimensions, so we recommend using `np.array`. `@` and `np.matmul`.

```
A = np.array([[1, 2, 3], [4, 5, 6]])
A.shape
# (2, 3)
B = np.array([[1, 2, 3, 4, 5],
              [6, 7, 8, 9, 10],
              [11, 12, 13, 14, 15]])
B.shape
# (3, 5)
np.dot(A, B).shape
# (2, 5)
np.dot(A, B)
# array([[ 46,  52,  58,  64,  70],
#        [100, 115, 130, 145, 160]])
```

Basic Operators

Transpose: if $A \in \mathbb{R}^{m \times n}$, then $A^T \in \mathbb{R}^{n \times m}$ and $(A^T)_{ij} = A_{ji}$.

- ▶ $(A^T)^T = A$
- ▶ $(AB)^T = B^T A^T$
- ▶ $(A + B)^T = A^T + B^T$

In Numpy:

```
A = np.array([[1, 2, 3], [4, 5, 6]])  
A  
# array([[1, 2, 3],  
#        [4, 5, 6]])  
  
A.T  
# array([[1, 4],  
#        [2, 5],  
#        [3, 6]])
```

Basic Operators (continued)

Trace: if $A \in \mathbb{R}^{n \times n}$, then

$$\text{tr}(A) = \sum_{i=1}^n A_{ii}.$$

► $\text{tr}(A + B) = \text{tr}(A) + \text{tr}(B)$

In Numpy:

```
A = np.array([[1, 2], [3, 4]])  
np.trace(A)  
# 5
```

Trace is invariant under any change of basis.

Basic Operators (continued)

A matrix $A \in \mathbb{R}^{n \times n}$ is **invertible** if $\exists B \in \mathbb{R}^{n \times n}$ such that

$$AB = BA = I.$$

B is called the inverse of A and is written $B = A^{-1}$.

- ▶ $(A^{-1})^{-1} = A$
- ▶ $(AB)^{-1} = B^{-1}A^{-1}$
- ▶ $(A^{-1})^T = (A^T)^{-1}$

In Numpy:

```
A = np.array([[1, 2], [3, 4]])
B = np.linalg.inv(A)
B
# array([[ -2. ,  1. ],
#        [ 1.5, -0.5]])

np.dot(A, B)
# array([[1., 0.],
#        [0., 1.]])
```

Basic Operators (continued)

What is the easiest way to check if a matrix A is invertible?

- Check $\det(A)$. If $\det(A) \neq 0$, then A is invertible.

In Numpy:

```
A = np.array([[1, 2], [3, 4]])  
np.linalg.det(A)  
# -2
```

Non-invertible matrices are also called **singular** or **degenerate**.

Invertibility in ML: math vs numerics

Mathematical fact. For square $A \in \mathbb{R}^{n \times n}$,

$$A \text{ invertible} \iff \det(A) \neq 0.$$

Numerical fact. Computing $\det(A)$ (or A^{-1}) is often *unstable* and unnecessary.

Best practice in code:

- ▶ Avoid explicitly forming A^{-1} .
- ▶ Solve linear systems with `np.linalg.solve(A, b)`.
- ▶ For least squares use `np.linalg.lstsq(A, b)` or SVD-based solvers.

Why avoid A^{-1} ? Errors amplify: if A is ill-conditioned, tiny perturbations in data yield large changes in the solution.

Avoid inv: stable NumPy patterns

```
# BAD (less stable, slower):  
x = np.linalg.inv(A) @ b  
  
# GOOD (solve  $Ax=b$ ):  
x = np.linalg.solve(A, b)  
  
# Least squares (min  $\|Ax-b\|_2$ ):  
x, *_ = np.linalg.lstsq(A, b, rcond=None)  
  
# Ridge regression (min  $\|Ax-b\|_2^2 + \|x\|_2^2$ ):  
lam = 1e-3  
x = np.linalg.solve(A.T@A + lam*np.eye(A.shape[1]), A.T@b)
```

Key idea: Often problems are often *rectangular* ($m \neq n$) or *rank-deficient*; inverse may not exist at all.

Invertibility: equivalent characterizations (linear algebra view)

Let $A \in \mathbb{R}^{n \times n}$. The following are equivalent:

1. A is invertible (exists A^{-1} with $AA^{-1} = A^{-1}A = I$).
2. $\det(A) \neq 0$.
3. $\text{rank}(A) = n$ (full rank).
4. $\mathcal{N}(A) = \{0\}$ (only solution to $Ax = 0$ is $x = 0$).
5. The columns of A are linearly independent (and form a basis of $\mathbb{R}^n$).
6. For every $b \in \mathbb{R}^n$, the system $Ax = b$ has a *unique* solution.
7. 0 is not an eigenvalue of A (equivalently all eigenvalues are nonzero).

How these fit together (proof sketches):

- ▶ (1) $\Rightarrow$ (4): If $Ax = 0$, multiply by A^{-1} to get $x = 0$.
- ▶ (4) $\Rightarrow$ (3): By rank-nullity, $\text{rank}(A) = n - \dim \mathcal{N}(A) = n$.
- ▶ (3) $\Rightarrow$ (6): full rank means columns span $\mathbb{R}^n$ and are independent.

Why $\det(A) = 0$ means “not invertible” (conceptual proof)

Key idea: $\det(A) = 0$ exactly means the columns of A are linearly dependent.

Proof sketch (via dependence $\Rightarrow$ nontrivial nullspace): If columns are dependent, then $\exists x \neq 0$ such that $Ax = 0$. So $\mathcal{N}(A) \neq \{0\}$, hence A cannot be invertible.

Proof sketch (via volume interpretation): For $A \in \mathbb{R}^{n \times n}$, $|\det(A)|$ equals the volume-scaling factor of the linear map $x \mapsto Ax$. If $\det(A) = 0$, volumes collapse to 0, meaning the image lies in a lower-dimensional subspace. So the map is not one-to-one, hence not invertible.

Connection to eigenvalues (optional intuition):

$$\det(A) = \prod_{i=1}^n \lambda_i \quad \Rightarrow \quad \det(A) = 0 \iff \exists i : \lambda_i = 0.$$

Existence/uniqueness for $Ax = b$ via rank (augmented matrix)

Let $A \in \mathbb{R}^{m \times n}$ and consider the system $Ax = b$. Define the augmented matrix $[A \ b] \in \mathbb{R}^{m \times (n+1)}$.

Rank criterion (general, works for rectangular systems):

- ▶ **Consistent (has at least one solution)** iff $\text{rank}(A) = \text{rank}([A \ b])$.
- ▶ If consistent, then the solution is **unique** iff $\text{rank}(A) = n$ (full column rank).
- ▶ If consistent and $\text{rank}(A) < n$, then there are **infinitely many** solutions.

Why this is true (proof sketch): Row-reduction does not change the solution set. In reduced form, a row of the form $[0 \ \cdots \ 0 \mid 1]$ appears *iff* b adds a new pivot, i.e. $\text{rank}([A \ b]) > \text{rank}(A)$, which makes the system inconsistent.

Rank: what's actually needed (not just det)

For $A \in \mathbb{R}^{m \times n}$, the **rank** is

$$\text{rank}(A) = \dim(\text{Col}(A)) = \dim(\text{Row}(A)).$$

Four fundamental subspaces:

- ▶ Column space $\text{Col}(A) = \{Ax : x \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$
- ▶ Null space $\mathcal{N}(A) = \{x : Ax = 0\} \subseteq \mathbb{R}^n$
- ▶ Row space $\text{Row}(A) = \text{Col}(A^T)$
- ▶ Left null space $\mathcal{N}(A^T)$

Rank-nullity theorem (proof sketch next slide):

$$\text{rank}(A) + \dim(\mathcal{N}(A)) = n.$$

Rank-nullity: conceptual proof sketch

Consider the linear map $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $T(x) = Ax$.

- ▶ Choose a basis $\{u_1, \dots, u_k\}$ for $\mathcal{N}(A)$ (so $Au_i = 0$).
- ▶ Extend it to a basis of $\mathbb{R}^n$:

$$\{u_1, \dots, u_k, v_1, \dots, v_r\}, \quad k + r = n.$$

- ▶ Then $\{Av_1, \dots, Av_r\}$ spans $\text{Col}(A)$.
- ▶ Show linear independence: if $\sum_{i=1}^r \alpha_i Av_i = 0$, then $A(\sum_i \alpha_i v_i) = 0$ so $\sum_i \alpha_i v_i \in \mathcal{N}(A)$. But the basis decomposition is unique, so all $\alpha_i = 0$.

Therefore $\dim(\text{Col}(A)) = r$ and $r + k = n$.

Solving $Ax = b$: when does a solution exist?

Let $A \in \mathbb{R}^{m \times n}$.

- ▶ **Exact solvability:** $Ax = b$ has a solution iff $b \in \text{Col}(A)$.
- ▶ **Uniqueness:** if a solution exists, it is unique iff $\mathcal{N}(A) = \{0\}$ (full column rank).
- ▶ **Overdetermined ($m > n$):** usually no exact solution; use least squares.
- ▶ **Underdetermined ($m < n$):** infinitely many solutions; often pick minimum-norm.

Interpretation:

- ▶ Overdetermined: regression with many data points.
- ▶ Underdetermined: overparameterized models, infinitely many interpolating solutions.

Special Matrices

If $A \in \mathbb{R}^{n \times n}$,

- ▶ A is **diagonal** if all its non-diagonal entries are zero:

$$\forall i \neq j : A_{ij} = 0.$$

- ▶ A is **symmetric** if $A = A^T$.
- ▶ A is **orthogonal** if $A^T = A^{-1}$ (i.e. $AA^T = A^T A = I$).

Positive (Semi)Definite matrices (PSD/PD)

A symmetric matrix $A = A^T \in \mathbb{R}^{n \times n}$ is:

- ▶ **Positive semidefinite (PSD)** if $\forall x, x^T A x \geq 0$.
- ▶ **Positive definite (PD)** if $\forall x \neq 0, x^T A x > 0$.

Equivalences (Spectral characterization): If A is symmetric, then

$$A \succeq 0 \iff \lambda_i(A) \geq 0 \ \forall i, \quad A \succ 0 \iff \lambda_i(A) > 0 \ \forall i.$$

Common examples: covariance matrices, Gram matrices $K = XX^T$, Hessians of convex quadratics.

Spectral theorem (needed for PCA, quadratics)

Theorem (Spectral). If $A = A^T \in \mathbb{R}^{n \times n}$, then there exists an orthogonal Q and diagonal Λ such that

$$A = Q\Lambda Q^T, \quad Q^T Q = I, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n).$$

Consequences:

- ▶ Quadratic form: $x^T A x = (Q^T x)^T \Lambda (Q^T x) = \sum_i \lambda_i z_i^2$.
- ▶ PSD is immediate: if $\lambda_i \geq 0$ then $x^T A x \geq 0$.
- ▶ Whitening / PCA use this diagonalization.

Proof sketch: symmetric matrices have an orthonormal eigenbasis; stack eigenvectors into Q .

Orthogonal matrices as distance-preserving transforms

$Q \in \mathbb{R}^{n \times n}$ is orthogonal if $Q^T Q = I$.

Key property: orthogonal transforms preserve inner products and norms:

$$\langle Qx, Qy \rangle = (Qx)^T (Qy) = x^T (Q^T Q) y = x^T y, \quad \|Qx\|_2 = \|x\|_2.$$

Interpretation: rotations/reflections. This is why PCA can be seen as an orthogonal change of coordinates.

Projections: the geometry behind least squares

A matrix $P \in \mathbb{R}^{m \times m}$ is a **projection** if $P^2 = P$. It is an **orthogonal projection** if additionally $P^T = P$.

Facts:

- ▶ If $P = P^T = P^2$, then $\text{Im}(P)$ and $\mathcal{N}(P)$ are orthogonal complements.
- ▶ For any b , Pb is the closest vector to b inside the subspace $\text{Im}(P)$:

$$Pb = \arg \min_{y \in \text{Im}(P)} \|y - b\|_2.$$

Least squares is exactly the orthogonal projection of b onto $\text{Col}(A)$.

Vector Spaces

- ▶ A vector space is a set of vectors that is closed under addition and scalar multiplication.
- ▶ Vector spaces also satisfy other axioms. Given a vector space V , for any $u, v, w \in V$:
- ▶ Associativity of addition: $u + (v + w) = (u + v) + w$
- ▶ Commutativity of addition: $u + v = v + u$
- ▶ Identity element of addition: $\exists 0 \in V$ such that $v + 0 = v$
- ▶ Inverse element of addition: $\exists (-v)$ such that $v + (-v) = 0$
- ▶ Associativity of scalar multiplication: $a(bv) = (ab)v$
- ▶ Identity element of multiplication: $1v = v$
- ▶ Distributivity: $a(u + v) = au + av$, $(a + b)v = av + bv$

$\mathbb{R}^n$ is a vector space.

Span, linear independence, basis, dimension

Given vectors $v_1, \dots, v_k \in \mathbb{R}^n$:

$$\text{span}\{v_1, \dots, v_k\} = \left\{ \sum_{i=1}^k \alpha_i v_i : \alpha_i \in \mathbb{R} \right\}.$$

Linear independence: $\sum_i \alpha_i v_i = 0 \Rightarrow \alpha_i = 0 \ \forall i$.

Basis: a linearly independent set that spans the space.

Dimension: number of vectors in any basis (well-defined).

Context: feature vectors span the hypothesis subspace; rank is the dimension of the span of columns of X .

Fundamental theorem of linear algebra (orthogonality)

For $A \in \mathbb{R}^{m \times n}$:

$$\text{Col}(A) \perp \mathcal{N}(A^T), \quad \text{Row}(A) \perp \mathcal{N}(A).$$

Reason: if $y \in \mathcal{N}(A^T)$, then for any x ,

$$\langle Ax, y \rangle = x^T A^T y = 0,$$

so y is orthogonal to every vector in $\text{Col}(A)$.

Least squares uses this: residual $r = b - Ax^*$ is orthogonal to $\text{Col}(A)$.

Vector Norm

A norm on a vector space V is a function $\|\cdot\| : V \rightarrow \mathbb{R}_+$ such that

- ▶ $\|x\| = 0$ iff $x = 0$
- ▶ $\|\alpha x\| = |\alpha| \|x\|$
- ▶ $\|x + y\| \leq \|x\| + \|y\|$

The norm of a vector is a measure of its magnitude. Formally,

- ▶ ℓ_p norm: $\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$.

The ℓ_2 and ℓ_1 norms, which are used in Ridge and Lasso regularization respectively, are the most common.

- ▶ ℓ_2 norm: $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$.
- ▶ ℓ_1 norm: $\|x\|_1 = \sum_{i=1}^n |x_i|$.

Vector Norm (continued)

In Numpy:

```
x = np.array([1, 2, 3])  
  
np.linalg.norm(x, ord=None)  
# 3.7416573867739413  
  
np.linalg.norm(x, ord=1)  
# 6.0
```

The `ord` parameter allows you to select what type of norm (ℓ_2 , ℓ_1 , etc.). It defaults to the Frobenius norm (2-norm).

Matrix norms (what shows up in generalization + stability)

A **matrix norm** $\|\cdot\|$ satisfies norm axioms and is submultiplicative:

$$\|AB\| \leq \|A\|\|B\|.$$

Induced (operator) 2-norm:

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2.$$

Frobenius norm:

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\text{tr}(A^T A)}.$$

Key identity: $\|A\|_2 = \sigma_{\max}(A)$ (largest singular value).

Condition number: the stability knob

For full-rank square A , the (2-norm) condition number is

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

Interpretation: how much relative error can be amplified in solving $Ax = b$.

Perturbation bound (informal but useful): If A is well-conditioned, solutions change mildly under small perturbations. If $\kappa(A)$ is huge, the problem is effectively near-singular.

Consequence: ill-conditioned $X^T X$ makes normal equations unstable; use SVD/QR or add ridge regularization.

Eigenvalues and Eigenvectors

Let $A \in \mathbb{R}^{n \times n}$. A scalar λ is called an **eigenvalue** of A if there exists $v \in \mathbb{R}^n$, $v \neq 0$, such that $Av = \lambda v$. Any vector $v \neq 0$ satisfying this equation is called an **eigenvector** associated with eigenvalue λ .

- ▶ We can rewrite the above equation as $(A - \lambda I_n)v = 0$.
- ▶ Strategy:
 - ▶ Find all λ for which the characteristic polynomial: $p_A(\lambda) = \det(A - \lambda I_n) = 0$.
 - ▶ For each λ , solve $(A - \lambda I_n)v = 0$ for v .

In Numpy:

```
A = np.array([[0.8, 0.3], [0.2, 0.7]])
np.linalg.eig(A)[0]
# array([1. , 0.5])
np.linalg.eig(A)[1]
# array([[ 0.83205029, -0.70710678],
#        [ 0.5547002 ,  0.70710678]])
```

SVD: the workhorse decomposition

Theorem (SVD). For any $A \in \mathbb{R}^{m \times n}$, there exist orthogonal $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ and diagonal $\Sigma \in \mathbb{R}^{m \times n}$ with nonnegative entries such that

$$A = U\Sigma V^T, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_r), \quad \sigma_1 \geq \dots \geq \sigma_r > 0,$$

where $r = \text{rank}(A)$.

Geometric meaning: V^T rotates, Σ scales along orthogonal axes, U rotates.

Eigenvalues vs singular values

If $A \in \mathbb{R}^{m \times n}$, then:

$$A^T A = V \Sigma^T \Sigma V^T,$$

so $A^T A$ is symmetric PSD and its eigenvalues are σ_i^2 .

Takeaways:

- ▶ Eigen-decomposition requires a *square* matrix; SVD works always.
- ▶ PCA uses SVD of centered data matrix X ; principal directions are columns of V .
- ▶ Conditioning: $\kappa_2(A) = \sigma_{\max} / \sigma_{\min}$.

Moore–Penrose pseudoinverse A^+ (handles rank deficiency)

Using SVD $A = U\Sigma V^T$, define

$$A^+ = V\Sigma^+U^T,$$

where Σ^+ is formed by inverting nonzero singular values: $\sigma_i \mapsto 1/\sigma_i$.

Why it matters:

- ▶ If A is square invertible, then $A^+ = A^{-1}$.
- ▶ If $Ax = b$ is inconsistent, $x^* = A^+b$ gives the **least-squares** minimizer.
- ▶ If solutions are non-unique, $x^* = A^+b$ is the **minimum ℓ_2 -norm** solution.

Least squares: projection + normal equations (fully justified)

Problem:

$$\min_x \|Ax - b\|_2^2.$$

Geometric statement: the optimal residual $r^* = b - Ax^*$ is orthogonal to $\text{Col}(A)$:

$$A^T(b - Ax^*) = 0.$$

Normal equations:

$$A^T A x^* = A^T b.$$

When is it unique? If A has full column rank ($\text{rank}(A) = n$), then $A^T A$ is PD and the solution is unique.

Warning: forming $A^T A$ can square the condition number; SVD/QR is more stable.

Matrix Calculus Outline

- ▶ The Gradient
- ▶ The Hessian Matrix
- ▶ Gradient for Vector-Valued Functions
- ▶ Handy Matrix Differentiation Rules
- ▶ Useful Resources
- ▶ Examples

The Gradient

Consider a differentiable real function $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$. For $X \in \mathbb{R}^{m \times n}$, the gradient $\nabla : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$ of f is by definition

$$\nabla_X f(X) = \begin{bmatrix} \frac{\partial f(X)}{\partial x_{11}} & \frac{\partial f(X)}{\partial x_{12}} & \dots & \frac{\partial f(X)}{\partial x_{1n}} \\ \frac{\partial f(X)}{\partial x_{21}} & \frac{\partial f(X)}{\partial x_{22}} & \dots & \frac{\partial f(X)}{\partial x_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f(X)}{\partial x_{m1}} & \frac{\partial f(X)}{\partial x_{m2}} & \dots & \frac{\partial f(X)}{\partial x_{mn}} \end{bmatrix}.$$

In most cases in this course, X will be a vector $x \in \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$. In this case, the gradient $\nabla : \mathbb{R}^n \rightarrow \mathbb{R}^n$ of f is

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}.$$

The Hessian Matrix

Let $x \in \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Then the Hessian matrix $\nabla^2 : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$ of f is

$$\nabla_x^2 f(x) = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(x)}{\partial x_2 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_2^2} & \cdots & \frac{\partial^2 f(x)}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \frac{\partial^2 f(x)}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{bmatrix}.$$

Gradient for Vector-Valued Functions

For $x \in \mathbb{R}$ and $v : \mathbb{R} \rightarrow \mathbb{R}^N$, the gradient $\nabla : \mathbb{R} \rightarrow \mathbb{R}^N$ of v is

$$\nabla_x v(x) = \begin{bmatrix} \frac{\partial v_1(x)}{\partial x} & \frac{\partial v_2(x)}{\partial x} & \dots & \frac{\partial v_N(x)}{\partial x} \end{bmatrix}.$$

For $x \in \mathbb{R}^K$ and $h : \mathbb{R}^K \rightarrow \mathbb{R}^N$, the gradient $\nabla : \mathbb{R}^K \rightarrow \mathbb{R}^{K \times N}$ of h (known as the Jacobian matrix) is

$$\nabla_x h(x) = \begin{bmatrix} \frac{\partial h_1(x)}{\partial x_1} & \frac{\partial h_2(x)}{\partial x_1} & \dots & \frac{\partial h_N(x)}{\partial x_1} \\ \frac{\partial h_1(x)}{\partial x_2} & \frac{\partial h_2(x)}{\partial x_2} & \dots & \frac{\partial h_N(x)}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_1(x)}{\partial x_K} & \frac{\partial h_2(x)}{\partial x_K} & \dots & \frac{\partial h_N(x)}{\partial x_K} \end{bmatrix}.$$

Gradient for Vector-Valued Functions (continued)

There are two main notational conventions in matrix calculus. For $x \in \mathbb{R}^K$ and $h : \mathbb{R}^K \rightarrow \mathbb{R}^N$:

Numerator layout: $\nabla : \mathbb{R}^K \rightarrow \mathbb{R}^{N \times K}$

$$\nabla_x h(x) = \begin{bmatrix} \frac{\partial h_1(x)}{\partial x_1} & \frac{\partial h_1(x)}{\partial x_2} & \dots & \frac{\partial h_1(x)}{\partial x_K} \\ \frac{\partial h_2(x)}{\partial x_1} & \frac{\partial h_2(x)}{\partial x_2} & \dots & \frac{\partial h_2(x)}{\partial x_K} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_N(x)}{\partial x_1} & \frac{\partial h_N(x)}{\partial x_2} & \dots & \frac{\partial h_N(x)}{\partial x_K} \end{bmatrix}.$$

Denominator layout: $\nabla : \mathbb{R}^K \rightarrow \mathbb{R}^{K \times N}$

$$\nabla_x h(x) = \begin{bmatrix} \frac{\partial h_1(x)}{\partial x_1} & \frac{\partial h_2(x)}{\partial x_1} & \dots & \frac{\partial h_N(x)}{\partial x_1} \\ \frac{\partial h_1(x)}{\partial x_2} & \frac{\partial h_2(x)}{\partial x_2} & \dots & \frac{\partial h_N(x)}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_1(x)}{\partial x_K} & \frac{\partial h_2(x)}{\partial x_K} & \dots & \frac{\partial h_N(x)}{\partial x_K} \end{bmatrix}.$$

You can use whichever you prefer, but be consistent.

Handy Matrix Differentiation Rules

Note that we are using **denominator layout**.

- ▶ $y = Ax$ where A does not depend on x :

$$\frac{\partial y}{\partial x} = A^T.$$

- ▶ $y = x^T A$ where A does not depend on x :

$$\frac{\partial y}{\partial x} = A.$$

- ▶ $y = x^T Ax$ where A does not depend on x :

$$\frac{\partial y}{\partial x} = x^T (A + A^T).$$

Matrix calculus: the differential method

To avoid layout confusion, use **differentials**.

Rule: For scalar $f(X)$,

$$df = \text{tr}((\nabla_X f)^T dX).$$

So: compute df in terms of dX , then read off the gradient.

Product rule (differential form):

$$d(XY) = (dX)Y + X(dY).$$

Trace cyclicity: $\text{tr}(ABC) = \text{tr}(BCA) = \text{tr}(CAB)$ (when dimensions match).

Trace trick identities

For compatible dimensions:

$$x^T y = \text{tr}(x^T y) = \text{tr}(y x^T), \quad \|Ax - b\|_2^2 = (Ax - b)^T (Ax - b).$$

Common gradients (unstructured case):

- ▶ $f(x) = a^T x \Rightarrow \nabla_x f = a.$
- ▶ $f(x) = x^T A x \Rightarrow \nabla_x f = (A + A^T)x.$
- ▶ $f(X) = \text{tr}(AX) \Rightarrow \nabla_X f = A.$
- ▶ $f(X) = \text{tr}(X^T B X) \Rightarrow \nabla_X f = (B + B^T)X.$

Important: these assume entries of X are independent (no symmetry constraint).

Structured matrices: why naive gradients can be wrong

If X is constrained (e.g., symmetric or diagonal), then not all entries are independent.

Example (symmetric constraint): if $X = X^T$, the correct update should respect symmetry. A common safe pattern in optimization is to symmetrize:

$$G \leftarrow \frac{1}{2}(G + G^T)$$

when the true variable is symmetric.

Takeaway: Always ask: *are the matrix entries independent, or is there structure?* The Matrix Cookbook explicitly warns about this distinction.

Differential of inverse and log-det

Let X be invertible. Then:

$$d(X^{-1}) = -X^{-1}(dX)X^{-1}.$$

Proof: Differentiate $XX^{-1} = I$:

$$dX \cdot X^{-1} + X \cdot d(X^{-1}) = 0 \Rightarrow d(X^{-1}) = -X^{-1}(dX)X^{-1}.$$

Log-determinant:

$$d(\log \det X) = \text{tr}(X^{-1}dX), \quad \Rightarrow \quad \nabla_X \log \det X = X^{-T}.$$

Used in: Gaussian log-likelihoods, covariance estimation, graphical models, information criteria.

Chain rule (vector form) with Jacobians

If $y = g(x)$ and $f = f(y)$, then

$$\nabla_x f = J_g(x)^T \nabla_y f,$$

where $J_g(x) = \frac{\partial y}{\partial x}$ is the Jacobian (remember denominator layout slides).

Example: $f(x) = \|Ax - b\|_2^2$ can be seen as $y = Ax - b$ then $f = \|y\|_2^2$:

$$\nabla_y f = 2y, \quad J_g(x) = A, \quad \Rightarrow \quad \nabla_x f = A^T(2y) = 2A^T(Ax - b).$$

We'll see this again in a later slide.

Addendum: “Trace is invariant under change of basis” (precise)

revisit: “*Trace is invariant under any change of basis.*”

Precise statement: trace is invariant under **similarity transforms**. If $B = S^{-1}AS$ for invertible S , then

$$\text{tr}(B) = \text{tr}(S^{-1}AS) = \text{tr}(ASS^{-1}) = \text{tr}(A).$$

Meaning: trace is basis-independent for linear operators (matrix representations differ by similarity).

Caution: trace is *not* invariant under arbitrary left/right multiplication (e.g., $\text{tr}(SA) \neq \text{tr}(A)$ in general).

Useful Resources

- ▶ All of the matrix calculus rules you need to know for this class can be found in *The Matrix Cookbook*.
- ▶ The Wikipedia article for matrix calculus is a very good introduction to this material.

Examples (Least Squares)

- Real-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ where for $x \in \mathbb{R}^n$,

$$f(x) = \|Ax - b\|_2^2 \quad (1)$$

$$= (Ax - b)^T (Ax - b) \quad (2)$$

$$= (x^T A^T - b^T)(Ax - b) \quad (3)$$

$$= x^T A^T Ax - b^T Ax - x^T A^T b + b^T b. \quad (4)$$

- To find the x that minimizes f , we need to compute the gradient with respect to x :

$$\nabla_x f(x) = \nabla_x (x^T A^T Ax - b^T Ax - x^T A^T b + b^T b) \quad (5)$$

$$= 2A^T Ax - 2A^T b. \quad (6)$$

Examples (Vector Valued Functions)

- ▶ Consider the vector-valued function $v : \mathbb{R} \rightarrow \mathbb{R}^K$ where for $x \in \mathbb{R}$,

$$v(x) = \langle x, x^2, \dots, x^n \rangle.$$

Then,

$$\nabla_x v(x) = \langle 1, 2x, \dots, nx^{n-1} \rangle.$$

- ▶ Generally and for most problems in this class, the gradients of matrix products will only require applying basic matrix calculus rules.